Universidad del Valle de Guatemala
Facultad de Ingeniería
CC3092 Deep Learning

Laboratorio 10 — Explicabilidad e Interpretabilidad (XAI)

Entrega parcial: Task 1 y Task 2

Integrante	Carné
Nicolle Gordillo	22246

Repositorio: https://github.com/nicollegordillo/Explicabilidad-e-Interpretabilidad-de-Modelos
Notebook: Lab10_XAI_22246.ipynb
Fecha: 16 de octubre de 2026

Configuración del experimento

Todo el código está en PyTorch, NumPy y scikit-learn. Grad-CAM y CAM se implementaron desde cero con *hooks* de PyTorch, sin pytorch-grad-cam ni captum; la librería shap se usó solo para TreeSHAP, KernelSHAP y sus gráficos, y el cálculo exacto de Shapley del Task 2.5 está escrito a mano. Se fijó la semilla aleatoria 42 en Python, NumPy, PyTorch, la partición de datos, el modelo y la selección del fondo. Todo se ejecutó en CPU (PyTorch 2.13.0, shap 0.49.1), por lo que los resultados son reproducibles salvo los tiempos de cómputo.

Task 1 — Grad-CAM y CAM sobre ResNet-18

Task 1.1 — Modelo, preprocesamiento e imágenes

Se cargó ResNet-18 de torchvision con los pesos IMAGENET1K_V1 en modo evaluación. Cada imagen se redimensionó para que su lado corto midiera 256 píxeles, se recortó el centro de 224 × 224 y se normalizó por canal con la media (0.485, 0.456, 0.406) y la desviación estándar (0.229, 0.224, 0.225) de ImageNet. Las tres imágenes provienen de Wikimedia Commons (Tabla 1).

Tabla 1. Imágenes utilizadas y su fuente.

Imagen	Caso	Autor y licencia (Wikimedia Commons)
(a) Banano	Objeto claro y centrado	Filo gèn', CC BY-SA 4.0 — File:Banana_on_whitebackground.jpg
(b) Perro y gato	Dos objetos de clases distintas	Peretz Partensky, CC BY-SA 2.0 — File:Dog_and_cat_showing_fight_or_flight_responses.jpg
(c) Vaca en la playa	Objeto fuera de contexto	Timothy A. Gonsalves, CC BY-SA 4.0 — File:Cow_Uthandi_Beach_Chennai_Aug21_D72_20734.jpg



Figura 1. Las tres imágenes después del preprocesamiento (224 × 224) con la clase de mayor probabilidad.

Tabla 2. Cinco clases de mayor probabilidad para cada imagen.

Imagen	Rango	Clase (índice ImageNet)	Probabilidad
(a)	1	banana (954)	0.9968
	2	lemon (951)	0.0019
	3	ram (348)	0.0003
	4	slug (114)	0.0002
	5	pitcher (725)	0.0002
(b)	1	Egyptian cat (285)	0.2329
	2	hyena (276)	0.1108
	3	bull mastiff (243)	0.0990
	4	Rhodesian ridgeback (159)	0.0746
	5	Great Dane (246)	0.0668
(c)	1	ox (345)	0.4210
	2	oxcart (690)	0.3408
	3	dingo (273)	0.0339
	4	Labrador retriever (208)	0.0138
	5	seashore (978)	0.0129

En la imagen (a) el modelo está prácticamente seguro (p = 0.9968 para *banana*). En la imagen (b) la probabilidad se reparte: el gato domina el top-1 con solo 0.2329 y el perro, que es un bóxer, no aparece como tal; su masa se distribuye entre razas grandes de pelo corto (*bull mastiff*, *Rhodesian ridgeback*, *Great Dane*) y *hyena*. Esto se explica en parte porque el recorte central corta el lomo del perro y su pose erguida es poco habitual. Como segunda clase de (b) se tomó la

clase de perro más probable, *bull mastiff* (índice 243, rango 3). En la imagen (c) el modelo acierta *ox* con 0.4210, pero asigna 0.3408 a *oxcart* y *seashore* aparece en el quinto lugar, lo que indica que el contexto de playa sí entra en la predicción.

El recuadro del objeto de la imagen (c), en coordenadas de la imagen de 224 × 224, es (x0, y0, x1, y1) = (14, 68, 154, 208): va de la grupa a la oreja de la vaca y de la punta de los cuernos a las pezuñas. Ocupa el 39.06 % del área de la imagen (Figura 2).

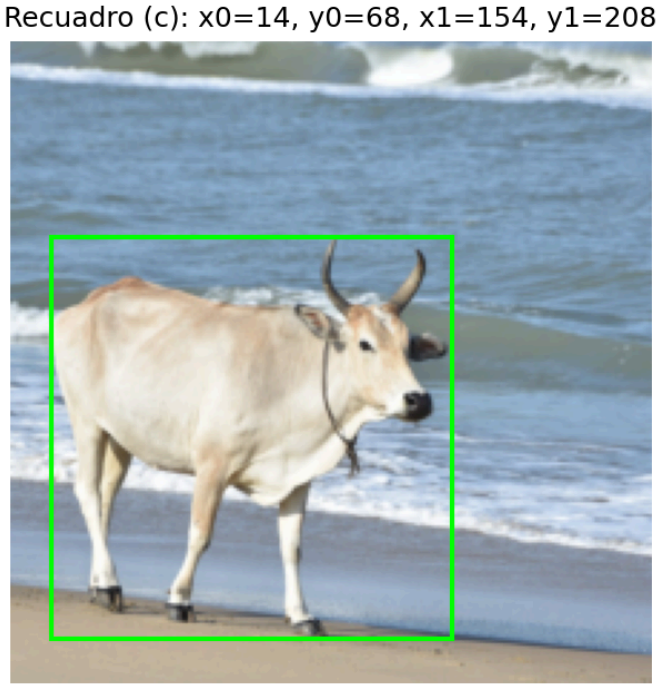


Figura 2. Recuadro del objeto en la imagen (c).

Task 1.2 — Grad-CAM desde cero

Se registró un *hook* de avance en layer4 que guarda las activaciones A (512 canales de 7 × 7) y un *hook* de retroceso (register_full_backward_hook) que guarda $\partial y^c/\partial A$. El gradiente se retropropaga desde el logit y^c, antes del softmax, con logits[0, c].backward(). Siguiendo la diapositiva de Grad-CAM, α_k^c es el promedio espacial del gradiente de cada canal, $\alpha_k^c = (1/Z)\,\Sigma_{ij}\,\partial y^c/\partial A^k_{ij}$ con $Z = 49$, y el mapa es $L^c = \mathrm{ReLU}(\Sigma_k\,\alpha_k^c\,A^k)$. Luego se amplía a 224 × 224 con interpolación bilineal y se normaliza con su mínimo y su máximo.

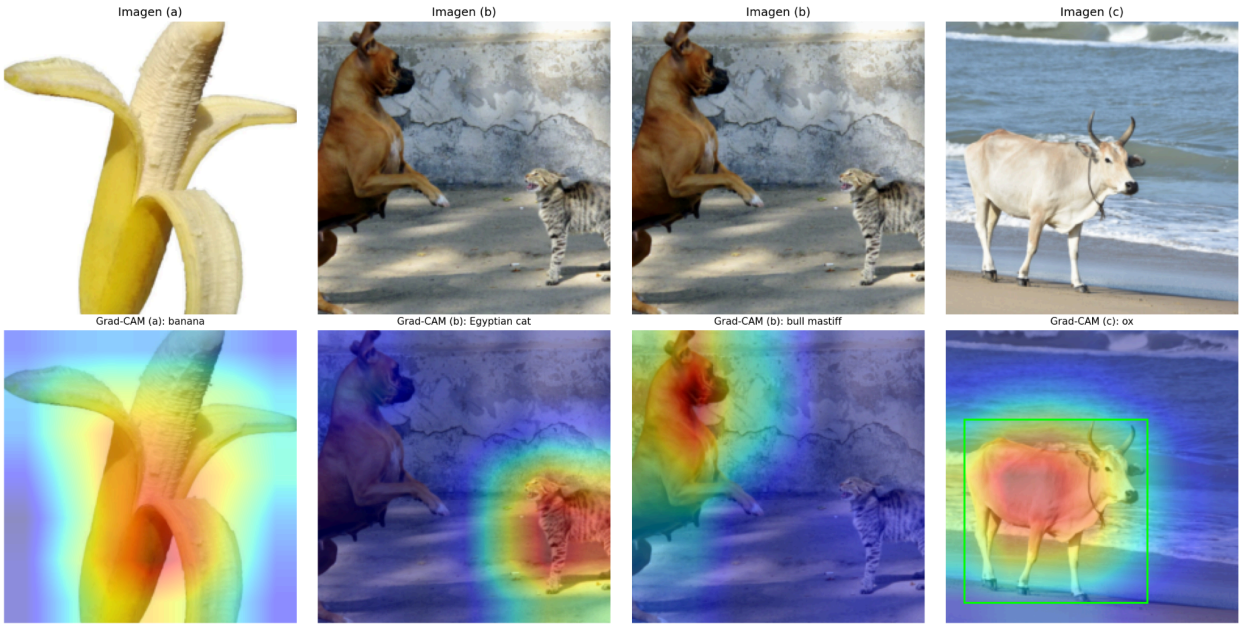


Figura 3. Grad-CAM de la clase de mayor probabilidad de cada imagen y de la segunda clase de (b).

En (a) el mapa se concentra sobre el cuerpo del banano, en la parte media e inferior donde está la fruta con la cáscara abierta, y es bajo en los bordes. En (b) los mapas de las dos clases caen sobre objetos distintos: *Egyptian cat* se concentra en el gato (abajo a la derecha) y *bull mastiff* en la cabeza y el pecho del perro (arriba a la izquierda). En (c) el mapa de *ox* cubre el cuerpo de la vaca dentro del recuadro y casi no activa el mar.

Task 1.3 — CAM y su relación con Grad-CAM

Como ResNet-18 termina en *global average pooling* y una capa lineal, el score de la clase es $y^c = \Sigma_k\, w_k^c\, z_k + b_c$ con $z_k = (1/Z)\, \Sigma_{ij}\, A^k_{ij}$ (diapositiva de CAM). El mapa CAM se calculó sin gradientes, con un solo *forward*, como $M^c_{ij} = \Sigma_k\, w_k^c\, A^k_{ij}$ usando los pesos de fc. La Tabla 3 verifica (b) la descomposición del puntaje $(1/Z)\, \Sigma_{ij}\, M^c_{ij} + b_c = y^c$, (c) la relación $Z \cdot L^c$ (sin ReLU ni normalización) $= M^c$ y (d) el error entre ambos mapas tras aplicar ReLU y normalización mínimo-máximo.

Tabla 3. Verificación numérica de CAM y de su relación con Grad-CAM (float32).

Mapa	Clase	Logit y^c	$(1/Z)\Sigma M + b_c$	(b) |error|	(c) máx $\|Z \cdot L - M\|$	(d) máx |error|
(a)	banana	18.670006	18.670008	1.9×10^{-6}	7.6×10^{-6}	3.0×10^{-7}
(b)	Egyptian cat	10.074001	10.074001	0	7.6×10^{-6}	1.8×10^{-7}
(b) 2.ª	bull mastiff	9.218128	9.218128	0	4.8×10^{-6}	1.8×10^{-7}
(c)	ox	10.182796	10.182796	0	1.1×10^{-5}	3.0×10^{-7}

La descomposición del puntaje se cumple con errores entre 0 y 1.9×10^{-6} sobre logits de 9 a 19, es decir, errores relativos del orden de 10^{-7}, que es la precisión de float32 ($\varepsilon \approx 1.19 \times 10^{-7}$). $Z \cdot L^c$ coincide con M^c con un error máximo de 1.1×10^{-5} en las 49 posiciones, y después de ReLU y normalización los dos mapas son idénticos hasta 3×10^{-7}. La Figura 4 muestra que los mapas de 7×7 de CAM y de Grad-CAM son visualmente iguales: en esta arquitectura Grad-CAM y CAM son el mismo mapa, salvo el factor Z.

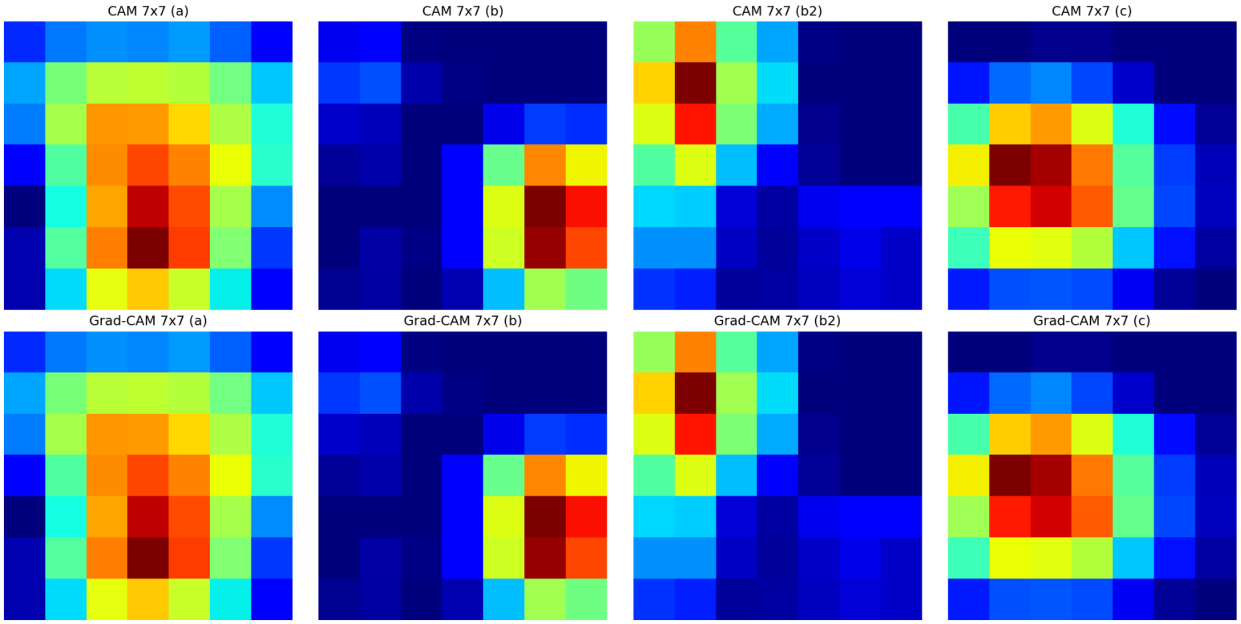


Figura 4. Mapas de 7×7 normalizados: CAM (fila superior) y Grad-CAM (fila inferior).

Task 1.4 — Preguntas

a. α_k^c frente a w_k^c/Z en la imagen (a). La Tabla 4 compara los cinco canales de mayor peso w_k^c para la clase *banana*. Además del cálculo normal en float32, se repitió el cálculo con una copia del modelo en float64.

Tabla 4. α_k^c de Grad-CAM frente a w_k^c/Z para los cinco canales de mayor peso, imagen (a).

Canal k	w_k^c	w_k^c / Z	α_k^c (float32)	Error rel. float32	Error rel. float64
171	0.332855	0.006792966	0.006792967	6.9×10^{-8}	1.3×10^{-16}
141	0.329215	0.006718673	0.006718673	6.9×10^{-8}	1.3×10^{-16}
496	0.302661	0.006176760	0.006176760	7.5×10^{-8}	0
41	0.301633	0.006155773	0.006155774	7.6×10^{-8}	0
226	0.300756	0.006137881	0.006137882	7.6×10^{-8}	1.4×10^{-16}

En los 512 canales el error relativo máximo es 2.3×10^{-7} en float32 y 2.5×10^{-16} en float64, en ambos casos del orden del épsilon de máquina (1.19×10^{-7} y 2.22×10^{-16}). Además, la

desviación estándar espacial del gradiente $\partial y^c/\partial A^k_{ij}$ dentro de cada canal es exactamente 0: el gradiente es el mismo en las 49 posiciones.

El paso de la derivación donde se usa que la derivada es igual en todas las posiciones es la derivada del *global average pooling*. Como $z_k = (1/Z)\,\Sigma_{ij}\, A^k_{ij}$ y $y^c = \Sigma_k\, w_k^c\, z_k + b_c$, por la regla de la cadena $\partial y^c/\partial A^k_{ij} = w_k^c \cdot \partial z_k/\partial A^k_{ij} = w_k^c/Z$, que no depende de (i, j) porque el promedio pesa por igual a todas las posiciones. Por eso, al promediar ese gradiente constante, $\alpha_k^c = (1/Z)\,\Sigma_{ij}\, w_k^c/Z = w_k^c/Z$, y en consecuencia $Z \cdot L^c = M^c$.

Los errores no son exactamente cero porque los dos lados se calculan con secuencias de redondeo distintas. w_k^c/Z es una sola división en punto flotante, mientras que α_k^c sale de la retropropagación (el gradiente pasa por fc y por avgpool, que divide entre 49) y luego de sumar 49 valores en float32 y dividir otra vez entre 49; cada operación redondea al número representable más cercano. Que el error baje nueve órdenes de magnitud al pasar a float64 confirma que la diferencia es solo de redondeo y no del método.

b. Discriminación entre clases en la imagen (b). Se tomó, en cada mapa de 224 × 224, el 20 % de píxeles de mayor valor. La intersección sobre la unión entre los conjuntos de *Egyptian cat* y *bull mastiff* es IoU = 0.000; con la rejilla original de 7 × 7 (10 de 49 celdas) también es 0.000, y la correlación de Spearman entre los dos mapas de 7 × 7 es −0.044 (Figura 5).

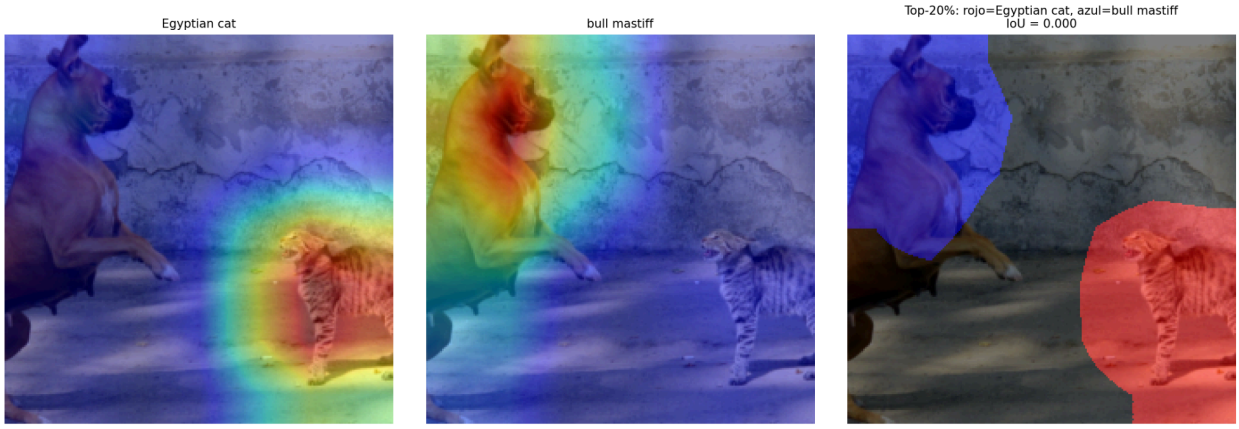


Figura 5. Imagen (b): Grad-CAM de cada clase y 20 % de píxeles de mayor valor (rojo: gato; azul: perro).

El mapa sí discrimina entre las dos clases: las regiones más importantes de cada una no comparten ningún píxel, y cada mapa cae sobre su objeto. Aquí funciona bien porque los dos animales están en lados opuestos de la imagen. Sin embargo, cada posición del mapa de 7 × 7 cubre un bloque de 32 × 32 píxeles de la entrada, y la interpolación bilineal solo suaviza el mapa, no le agrega resolución. Por eso Grad-CAM sobre layer4 no puede localizar objetos más pequeños que unos 32 píxeles: un objeto así aparece como una mancha de al menos una celda, más grande que el objeto real. Tampoco puede separar dos objetos que estén a menos de una celda de distancia: sus activaciones caen en las mismas celdas, los mapas de ambas clases se superponen y el IoU subiría aunque el modelo distinga las clases.

c. Uso del fondo en la imagen (c). El 74.5 % de la masa del mapa normalizado de *ox* cae dentro del recuadro, que ocupa solo el 39.1 % del área; un mapa uniforme tendría 39.1 %, así que el mapa se concentra en la vaca 1.9 veces más que el azar. Para la prueba causal se reemplazó por gris medio (0.5 por canal) todo lo que queda fuera del recuadro y, como prueba complementaria, todo lo que queda dentro (Tabla 5 y Figura 6).

Tabla 5. Prueba causal sobre la imagen (c).

Entrada	p(ox)	Cambio	Clase top-1
Imagen original	0.4210	—	ox (0.421)
Fondo en gris (solo el objeto)	0.2861	−0.1349	oxcart (0.411)
Objeto en gris (solo el fondo, extra)	0.0001	−0.4210	seashore (0.326)



Figura 6. Imagen (c): Grad-CAM original, versiones con el fondo y con el objeto en gris, y Grad-CAM con el fondo en gris.

El mapa y la prueba causal coinciden: el modelo usa principalmente a la vaca. Con solo el objeto, la probabilidad de *ox* conserva el 68 % de su valor (0.286 de 0.421); con solo el fondo cae a 0.0001 y la predicción pasa a *seashore*, de modo que la playa por sí sola no sostiene la clase. El fondo sí modula la decisión: al quitarlo, p(ox) baja 0.135 y el top-1 cambia a *oxcart*, una clase muy cercana. Ese efecto también es coherente con el 25.5 % de la masa del mapa que cae fuera del recuadro, que en parte se debe a la resolución de 32 píxeles en los bordes del objeto. Hay que considerar que el parche gris también saca a la imagen de la distribución de entrenamiento, lo que puede bajar la probabilidad por sí solo.

Conclusión: no hay evidencia de un atajo por el fondo; la clase depende sobre todo del objeto, y el contexto inusual de la playa más bien le resta confianza (0.42) y favorece clases vecinas como *oxcart* y *seashore*.

Task 2 — SHAP sobre un modelo de crédito

Task 2.1 — Datos y modelo

Se descargó German Credit con fetch_openml("credit-g", version=1): 1000 solicitudes con 20 variables (7 numéricas y 13 categóricas). La clase *good* se codificó como 1 y *bad* como 0; el 70

% de las solicitudes son *good*. Las variables categóricas se codificaron con OrdinalEncoder, que asigna los códigos 0, 1, 2, … en orden alfabético de las categorías.

Limitación de la codificación ordinal. Los códigos son arbitrarios, pero el gradient boosting los trata como números y hace cortes del tipo «código ≤ t». Eso impone un orden que no existe (por ejemplo, entre los propósitos del crédito) y obliga a los árboles a agrupar categorías que solo son vecinas por orden alfabético. Por eso, en los gráficos de SHAP el color de una variable categórica (valor alto o bajo) no tiene un significado ordinal y debe leerse por categoría.

Los datos se dividieron en 80 % de entrenamiento (800) y 20 % de prueba (200), de forma estratificada, y se entrenó un GradientBoostingClassifier con 200 árboles, profundidad máxima 3 y semilla 42. En el conjunto de prueba se obtuvo una exactitud de 0.775 y un AUC de 0.765. Como el 70 % de los clientes son *good*, la exactitud está solo 7.5 puntos por encima de aprobar a todos, así que el AUC es la métrica más informativa.

Task 2.2 — TreeSHAP intervencional

Se calcularon los valores SHAP del conjunto de prueba con TreeExplainer en su versión intervencional, sobre la salida en log-odds (model_output="raw", igual a decision_function) y con un fondo de 100 instancias de entrenamiento elegidas al azar con la semilla 42. Según la diapositiva de SHAP, la atribución es aditiva, $f(x) = \varphi_0 + \Sigma_i \varphi_i$, con $\varphi_0 = E[f(X)]$.

a. Eficiencia. Para las 200 instancias de prueba, el error absoluto máximo de $|\varphi_0 + \Sigma_j \varphi_j - f(x)|$ es 9.7×10^{-8}, de nuevo del orden de la precisión numérica.

b. Valor base. El valor base es $\varphi_0 = 1.430742$ y el promedio de f(x) sobre las 100 instancias del fondo es 1.430742; la diferencia es exactamente 0. En probabilidad, $\sigma(\varphi_0) = 0.807$, que es la tasa de aprobación promedio que el modelo predice para el fondo.

c. Importancia global. La Tabla 6 compara la importancia SHAP $I_j = (1/N) \Sigma_n |\varphi_j^{(n)}|$ con la importancia por permutación de scikit-learn (caída del AUC en prueba, 10 repeticiones).

Tabla 6. Cinco variables principales según SHAP y según permutación.

Rango	Variable (SHAP)	I_j	Variable (permutación)	Caída de AUC
1	checking_status	0.760	checking_status	0.119 ± 0.030
2	purpose	0.512	credit_amount	0.076 ± 0.014
3	credit_amount	0.490	duration	0.058 ± 0.012
4	duration	0.392	credit_history	0.040 ± 0.011
5	savings_status	0.296	savings_status	0.026 ± 0.011

Cuatro de las cinco variables coinciden (*checking_status*, *credit_amount*, *duration* y *savings_status*), y el estado de la cuenta corriente es la más importante en ambos. Las diferencias principales son *purpose*, segunda para SHAP pero duodécima por permutación, y *credit_history*, cuarta por permutación pero séptima para SHAP. La correlación de Spearman entre los dos

rankings completos de 20 variables es 0.650: hay acuerdo moderado, pero no son intercambiables, porque miden cosas distintas (se discute en el Task 2.6b).

d. Explicaciones locales. Se eligió, entre los clientes de prueba con p(good) > 0.8 (93 clientes), el de menor probabilidad, y entre los de p(good) < 0.2 (15 clientes), el de mayor probabilidad; es decir, los más cercanos a cada umbral. El cliente aprobado es el índice 133 (p = 0.811) y el rechazado es el índice 876 (p = 0.194). Este último se reutiliza en los Tasks 2.6, 3 y 4. Ambos son *good* en la realidad, así que el rechazado es un falso negativo del modelo.

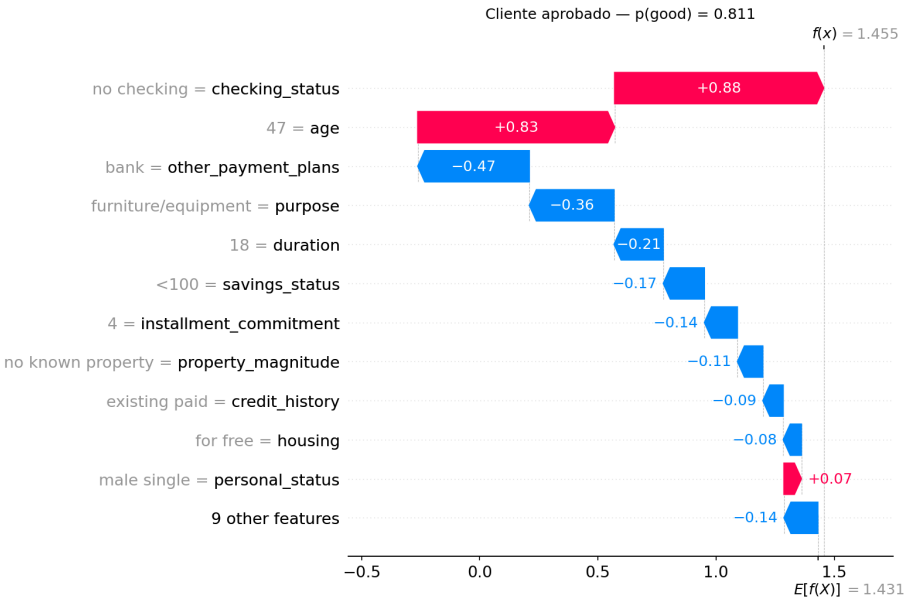


Figura 7. Cascada SHAP del cliente aprobado (índice 133, p(good) = 0.811).

Cliente aprobado. Parte del valor base de 1.431 log-odds y termina en f(x) = 1.455. Sus tres contribuciones más grandes son: (1) *checking_status* = «no checking» (+0.88): no tener cuenta corriente con el banco es, en estos datos, la señal más fuerte de buen pagador; (2) *age* = 47 años (+0.83): la edad madura sube la probabilidad de pago; y (3) *other_payment_plans* = «bank» (−0.47): tener otros planes de pago con un banco baja su probabilidad. El resultado neto queda cerca del valor base porque varias contribuciones negativas pequeñas, como el propósito «furniture/equipment» (−0.36) y la duración de 18 meses (−0.21), compensan los dos efectos positivos.

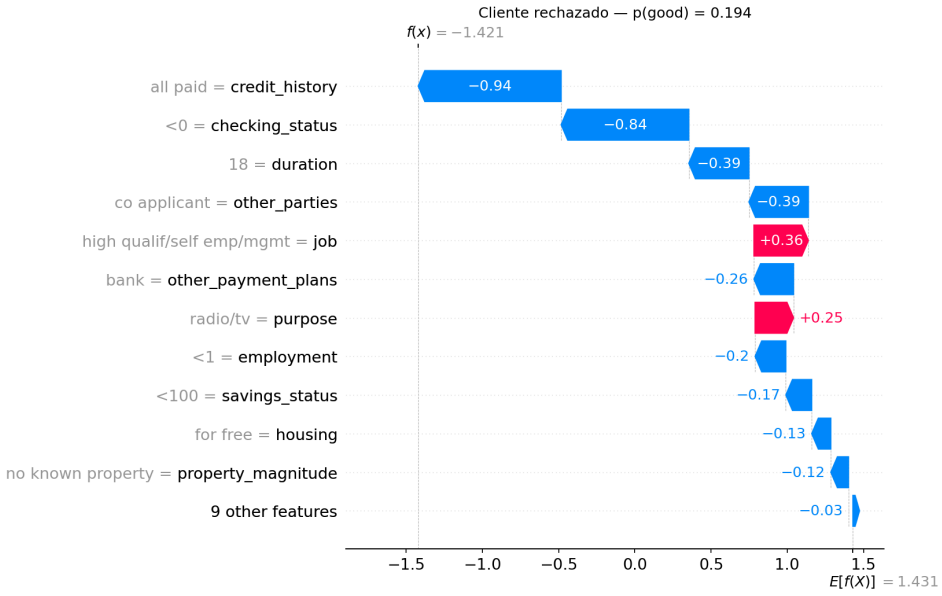


Figura 8. Cascada SHAP del cliente rechazado (índice 876, p(good) = 0.194).

Cliente rechazado. Parte de 1.431 y termina en f(x) = −1.421 log-odds. Sus tres contribuciones más grandes son todas negativas: (1) *credit_history* = «all paid» (−0.94): que todos sus créditos en este banco estén pagados le resta, un efecto contraintuitivo pero conocido en German Credit, donde esa categoría tiene más clientes *bad* que el historial «existing paid»; (2) *checking_status* = «<0» (−0.84): un saldo negativo en la cuenta corriente es la señal más clara de riesgo; y (3) *duration* = 18 meses (−0.39): la duración del crédito le resta. A favor solo tiene su empleo altamente calificado (+0.36) y el propósito «radio/tv» (+0.25).

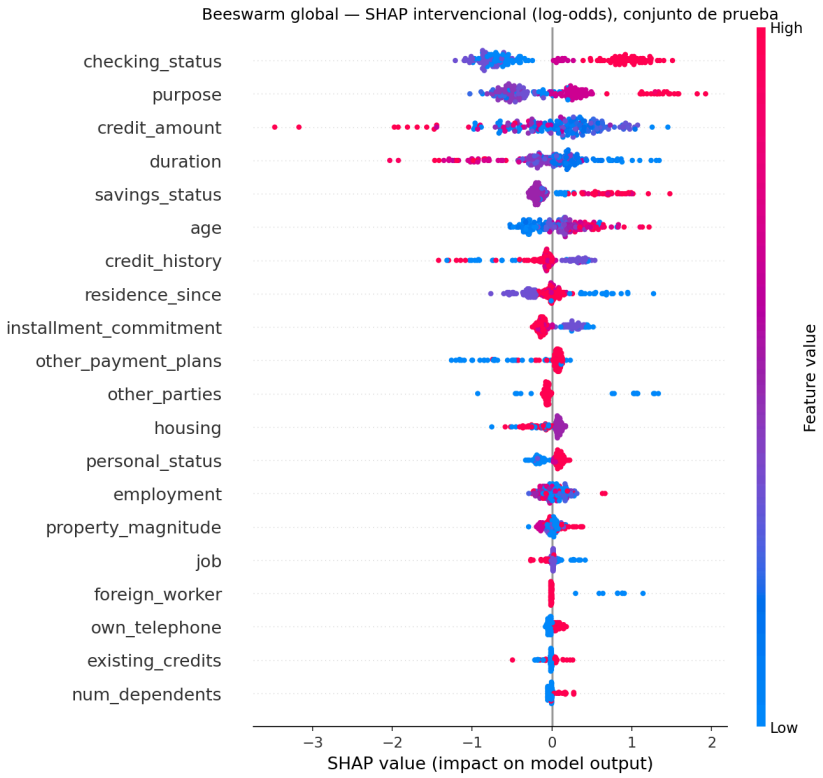


Figura 9. Gráfico beeswarm de SHAP sobre el conjunto de prueba (log-odds).

En el beeswarm global, *checking_status* separa a los clientes en dos grupos claros, uno con contribuciones positivas y otro negativas. Para las variables numéricas el color sí tiene un orden: montos de crédito y duraciones altas (rojo) empujan hacia *bad*, y la edad alta empuja hacia *good*. Para las categóricas el color refleja el código ordinal y no tiene un significado de «más» o «menos».

Task 2.3 — SHAP con una variable redundante

La variable numérica de mayor I_j es credit_amount ($I_j = 0.490$, $\sigma_x = 2822.7$). Se agregó la copia $x_{dup} = x + 0.05\ \sigma_x\ \varepsilon$ con $\varepsilon \sim N(0, 1)$, cuya correlación de Pearson con la original es 0.99878, y se reentrenó el modelo con 21 variables, las mismas filas de entrenamiento y prueba y la misma semilla (exactitud 0.760, AUC 0.767). La Tabla 7 resume los valores SHAP con las dos configuraciones.

Tabla 7. Promedios de $|\varphi|$ de credit_amount y su copia; la última columna es la misma variable en el modelo sin copia y con la misma configuración.

Configuración	mean $\lvert\varphi_{orig}\rvert$	mean $\lvert\varphi_{copia}\rvert$	mean $\lvert\varphi_{orig} + \varphi_{copia}\rvert$	I_j sin copia
Intervencional (fondo 100)	0.518	0.428	0.546	0.490
tree_path_dependent	0.381	0.363	0.531	0.474

Como evidencia adicional, en los 200 árboles del modelo con copia la variable original se usa en 156 cortes y la copia en 210 (366 en total), frente a 285 cortes de la variable en el modelo sin copia. La correlación entre φ_{orig} y φ_{copia} en la configuración intervencional es −0.30.

Task 2.4 — KernelSHAP frente a TreeSHAP

Se calcularon con KernelExplainer los valores SHAP de las 10 primeras instancias de prueba, sobre la salida en log-odds y con 2000 muestras por instancia, para fondos de 10, 50 y 100 instancias (los fondos de 10 y 50 son los primeros elementos del fondo de 100). Se usó l1_reg=False para no seleccionar variables. Cada resultado se comparó con TreeSHAP intervencional calculado con el mismo fondo (Tabla 8).

Tabla 8. Error de KernelSHAP respecto a TreeSHAP con el mismo fondo y tiempos de cómputo para 10 instancias.

Fondo	MAE	Error máximo	Tiempo Kernel (s)	Kernel por instancia (s)	Tiempo Tree (s)
10	0.0024	0.017	0.874	0.087	0.013
50	0.0049	0.048	2.668	0.267	0.026
100	0.0044	0.044	5.162	0.516	0.042

KernelSHAP reproduce los valores de TreeSHAP con un error absoluto medio de entre 0.002 y 0.005 log-odds, pequeño frente a contribuciones típicas de 0.1 a 1, pero es entre 69 y 123 veces más lento que TreeSHAP.

Task 2.5 — Shapley exacto escrito a mano

Se entrenó un segundo modelo con las mismas condiciones, pero solo con las seis variables numéricas *duration*, *credit_amount*, *installment_commitment*, *residence_since*, *age* y *existing_credits* (AUC de prueba 0.634), y se usó como fondo las primeras 50 instancias del fondo del Task 2.2. Para cada una de las 5 primeras instancias de prueba:

- **v(S)** es el promedio de la salida en log-odds sobre las 50 filas del fondo, tomando de la instancia los valores de las variables en S y del fondo los de las demás. Cada coalición se representa como un entero de 6 bits, y las 64 coaliciones se evalúan en una sola predicción de 64 × 50 filas.
- $\boldsymbol{\varphi_i} = \Sigma_{S \subseteq N\setminus\{i\}} |S|!\,(n - |S| - 1)! / n! \cdot [v(S \cup \{i\}) - v(S)]$, con n = 6. Como en la diapositiva de Shapley, el peso es la fracción de los n! órdenes de llegada en los que llegan primero exactamente las variables de S y luego i.
- Como verificación extra, φ_i también se calculó como el promedio del aporte marginal de i sobre los 6! = 720 órdenes de llegada.

Tabla 9. Valores de Shapley exactos (log-odds) del modelo de seis variables.

Variable	Inst. 0	Inst. 1	Inst. 2	Inst. 3	Inst. 4
duration	−0.5530	−1.6542	−0.0668	0.3526	0.3891
credit_amount	0.1944	0.1453	−1.5555	0.5675	−1.6156
installment_commitment	−0.0729	−0.0403	0.0535	−0.0354	0.0269
residence_since	−0.1193	−0.1095	0.0500	−0.1535	0.0073
age	0.2395	−0.3313	0.5447	0.3985	0.9232
existing_credits	0.1092	−0.0879	−0.0986	−0.1358	−0.1045
$\Sigma\ \varphi_i$	**−0.2022**	**−2.0779**	**−1.0727**	**0.9939**	**−0.3736**
$v(N) - v(\emptyset)$	**−0.2022**	**−2.0779**	**−1.0727**	**0.9939**	**−0.3736**

El error máximo entre los φ_i calculados a mano y los de TreeSHAP intervencional con el mismo fondo es 2.5×10^{-8} en las 30 combinaciones de instancia y variable. La eficiencia se cumple con error máximo 4.4×10^{-16}: $\Sigma\ \varphi_i = v(N) - v(\emptyset)$, con $v(N) = f(x)$ (error 5.6×10^{-17}) y $v(\emptyset) = 1.5447$, que coincide exactamente con el valor base de TreeSHAP. El cálculo por los 720 órdenes da los mismos valores con un error de 9.3×10^{-15}. Esto confirma que TreeSHAP intervencional calcula exactamente los valores de Shapley del juego v(S) definido con el fondo.

Task 2.6 — Preguntas

a. Log-odds frente a probabilidad. Para el cliente rechazado, $p(x) = \sigma(f(x)) = 0.1944$ y la probabilidad base es $\sigma(\varphi_0) = 0.8070$. La Tabla 10 muestra el efecto individual en probabilidad $\sigma(\varphi_0 + \varphi_j) - \sigma(\varphi_0)$ de las seis variables con mayor $|\varphi_j|$.

Tabla 10. Efectos individuales en probabilidad del cliente rechazado (seis mayores de 20).

Variable	Valor	φ_j (log-odds)	Efecto en probabilidad
credit_history	all paid	−0.938	−0.186
checking_status	<0	−0.837	−0.163
duration	18	−0.394	−0.069
other_parties	co applicant	−0.390	−0.068
job	high qualif/self emp/mgmt	+0.357	+0.050
other_payment_plans	bank	−0.260	−0.044

La suma de los 20 efectos individuales en probabilidad es −0.5494, mientras que $p(x) - \sigma(\varphi_0) = -0.6126$; la diferencia es 0.063. En cambio, en log-odds la suma es exacta: $\varphi_0 + \Sigma\ \varphi_j = -1.4214 = f(x)$. Los valores SHAP suman exactamente en log-odds porque el juego v(S) se definió sobre f, la salida en log-odds, y el axioma de eficiencia garantiza $\Sigma\ \varphi_j = f(x) - \varphi_0$ para esa función. La

sigmoide no es lineal, así que $\sigma(\varphi_0 + \Sigma\ \varphi_j) - \sigma(\varphi_0) \neq \Sigma\ [\sigma(\varphi_0 + \varphi_j) - \sigma(\varphi_0)]$. Cada efecto individual se mide desde $\varphi_0 = 1.43$, donde la pendiente de la sigmoide es $\sigma'(\varphi_0) = 0.156$, pero la suma de todas las contribuciones lleva al cliente a −1.42 pasando por la zona alrededor de 0, donde la pendiente llega a 0.25. Por eso la suma de los efectos aislados subestima la caída real de la probabilidad. El signo y el orden de las variables se conservan porque σ es monótona, pero sus magnitudes ya no se suman.

b. Diferencia entre los rankings. La variable con mayor diferencia de posición es age: es sexta por SHAP ($I_j = 0.289$) y vigésima, la última, por permutación (caída de AUC de -0.0054 ± 0.0096, indistinguible de cero). Para formular una hipótesis se revisaron dos cosas:

- **Correlación con otras variables.** La correlación de Spearman de *age* con las demás variables en entrenamiento es moderada como máximo: *housing* −0.31, *employment* 0.31, *personal_status* 0.27, *residence_since* 0.24 y *credit_history* −0.20. Parte de su información está repetida en otras variables, lo que reduce la caída al barajarla, pero no al punto de explicar una caída nula.
- **Si la señal generaliza.** Al barajar *age* con log-loss como métrica, la pérdida también mejora ligeramente (−0.016) y vuelve a quedar en el último lugar. Al mismo tiempo, su φ varía mucho entre clientes (desviación estándar de 0.345 log-odds, 53 valores distintos), aunque solo 8 de 200 clientes de prueba cambiarían de decisión si se quitara su contribución.

Hipótesis: el modelo usa la edad con contribuciones grandes al log-odds, pero esa relación aprendida en entrenamiento no mejora el orden de los clientes en prueba: es en buena parte sobreajuste, y la información que sí sirve está parcialmente repetida en *housing*, *employment* y *personal_status*. Las dos importancias miden cosas distintas. La permutación mide cuánto empeora una métrica de desempeño, en este caso el AUC, cuando se rompe la relación entre la variable y la etiqueta; si la variable no aporta poder predictivo que generalice, la caída es cero aunque el modelo la use mucho. I_j mide el tamaño promedio de la contribución de la variable al log-odds, es decir, cuánto mueve la salida del modelo, sin importar si ese movimiento acierta o se equivoca. Por eso *age* puede mover mucho la salida sin mejorar el AUC.

c. Crédito de la variable duplicada. Según la Tabla 7, en la configuración intervencional el crédito de *credit_amount* se reparte de forma desigual entre la original (0.518) y la copia (0.428). La suma de los $|\varphi|$ individuales (0.946) casi duplicaría la importancia real, pero $|\varphi_{orig} + \varphi_{copia}| = 0.546$ queda cerca del I_j de 0.490 que la variable tenía sola: el crédito total se conserva aproximadamente y lo que cambia es cómo se reparte.

El axioma que repartiría el crédito por igual es el de simetría: si dos variables aportan lo mismo a toda coalición, $v(S \cup \{i\}) = v(S \cup \{j\})$, reciben el mismo φ. No se cumple en este modelo porque el axioma se refiere a la función f, no a los datos. Los árboles son codiciosos: en cada nodo eligen la columna que da una ganancia ligeramente mayor, y como la copia tiene un 5 % de ruido, a veces gana la original y a veces la copia (156 frente a 210 cortes). La función resultante

usa las dos variables de forma distinta, así que para el modelo no son jugadores simétricos. La correlación de −0.30 entre φ_{orig} y φ_{copia} muestra que, en parte, se compensan entre sí.

Con *tree_path_dependent* el reparto es más parejo (0.381 frente a 0.363) y la suma (0.531) también se acerca al valor sin copia (0.474). Esta configuración no es exactamente la versión condicional. Para estimar $E[f(X) \mid X_S = x_S]$, cuando un nodo corta por una variable que no está en S, sigue ambas ramas ponderadas por la fracción de muestras de entrenamiento que pasó por cada una (*cover*). Solo condiciona por los cortes del camino del árbol: si la original está en S y el nodo corta por la copia, que no está en S, reparte el peso entre ambas ramas como si la copia fuera independiente, aunque con la original fija la copia es prácticamente igual. La expectativa condicional verdadera mandaría casi toda la masa por la rama consistente con el valor conocido.

Implicación: un valor SHAP mide la importancia de una variable para el modelo, no en el mundo real. Con variables correlacionadas o duplicadas, el crédito se reparte de forma arbitraria según detalles del entrenamiento, y una variable puede recibir un φ bajo solo porque una gemela se quedó con el crédito. No se puede concluir que el monto del crédito «importa 0.43» en la realidad; para hablar de importancia real hay que agrupar las variables correlacionadas o usar un análisis causal.

d. Costo computacional. El modelo de costo propuesto para KernelSHAP es: evaluaciones del modelo ≈ muestras por instancia × tamaño del fondo, es decir, 20 000, 100 000 y 200 000 filas para fondos de 10, 50 y 100, una proporción 1 : 5 : 10. Los tiempos medidos están en proporción 1 : 3.05 : 5.91: crecen de forma aproximadamente lineal con el fondo, pero con un costo fijo. Un ajuste lineal $t \approx 0.40\ s + 0.048\ s \times B$ reproduce los tres puntos (predice 2.78 s para B = 50 frente a 2.67 s medidos). El costo fijo corresponde a muestrear las coaliciones, armar las matrices y resolver la regresión ponderada, que no dependen de B, y además predecir en lotes grandes es más eficiente por fila.

El error no baja al aumentar el fondo (MAE de 0.0024, 0.0049 y 0.0044). Es lo esperado, porque la comparación es contra TreeSHAP calculado con el mismo fondo: el fondo define el juego v(S) y no es una fuente de error en esta comparación. El error de KernelSHAP viene de usar solo 2000 de las $2^{20} \approx 1.05$ millones de coaliciones, y depende del número de muestras, no del fondo. Un fondo más grande sí reduciría el error de estimar E[f(X)] respecto a la distribución real de los datos, pero eso es otro tipo de error que esta comparación no mide.

Una predicción sobre 100 instancias tarda 1.168 ms (mediana de 50 repeticiones). El cálculo exacto de los Shapley de una instancia del modelo de 20 variables requiere $2^{20} = 1\ 048\ 576$ evaluaciones de v(S), cada una con una predicción sobre las 100 filas del fondo: $2^{20} \times 1.168\ ms \approx 1224\ s \approx 20.4$ minutos por instancia. KernelSHAP con el mismo fondo tarda 0.516 s por instancia, unas 2372 veces menos, a cambio de un error medio de 0.004 log-odds. Con 60 variables y un microsegundo por evaluación, el cálculo exacto tomaría $2^{60}\ \mu s \approx 1.15 \times 10^{12}\ s \approx 36\ 534$ años, lo que muestra por qué el cálculo exacto de 2^n coaliciones es inviable con muchas variables y hacen falta TreeSHAP o KernelSHAP.

Task 3 — LIME y explicaciones contrafactuales

Task 3.1 — LIME tabular desde cero

LIME se implementó sin la librería lime, sobre el gradient boosting del Task 2.1, siguiendo la diapositiva de LIME: se generan muestras perturbadas alrededor de x, se ponderan por cercanía π_x y se ajusta un modelo lineal g fiel solo cerca de x. La representación interpretable usa las 20 variables originales: cada vector binario tiene 20 componentes iguales a 1 con probabilidad 0.5; si la componente es 1 la variable conserva el valor de la instancia, y si es 0 se reemplaza por el valor de una fila de entrenamiento elegida al azar (una fila distinta para cada variable ausente). Se usaron N = 5000, K = 5, $\lambda = 1.0$ y $\sigma_{rel} = 0.75$.

- La salida a explicar es p(good). El peso de cada muestra es $\pi_x(z) = \exp(-D^2/\sigma^2)$, con D^2 el número de variables ausentes y $\sigma = \sigma_{rel} \cdot \sqrt{20} = 3.354$.
- La regresión ponderada se resolvió con la forma cerrada $w = (Z^T \Pi Z + \lambda I)^{-1} Z^T \Pi y$, agregando una columna de unos. El intercepto no se penalizó: la entrada de λI que le corresponde es 0, para que la regularización no lo empuje hacia 0 y g pueda centrarse en el nivel de p(good).
- Se conservaron las K = 5 variables de mayor |w| y se volvió a ajustar solo con ellas; la fidelidad es $R^2_w = 1 - \Sigma_n \pi_n (y_n - g_n)^2 / \Sigma_n \pi_n (y_n - \bar{y}_w)^2$, con $\bar{y}_w = \Sigma_n \pi_n y_n / \Sigma_n \pi_n$.

Se aplicó a tres clientes de prueba: el aprobado del Task 2.2 (p > 0.8), el rechazado del Task 2.2 y, entre los 10 clientes con p(good) entre 0.45 y 0.55, el más cercano a 0.5 (índice 164, p = 0.504). Los tres son *good* en la realidad.

Tabla 11. Coeficientes de LIME de las K = 5 variables de cada cliente (cambio en p(good) cuando la variable conserva el valor del cliente).

Cliente	Variable = valor	w (K = 5)
Aprobado (índice 133)	age = 47	+0.1433
p(good) = 0.811	checking_status = no checking	+0.1277
	other_payment_plans = bank	−0.0845
	savings_status = <100	−0.0370
	purpose = furniture/equipment	−0.0351
Rechazado (índice 876)	credit_history = all paid	−0.1642
p(good) = 0.194	checking_status = <0	−0.1640
	other_parties = co applicant	−0.0615
	purpose = radio/tv	+0.0504
	job = high qualif/self emp/mgmt	+0.0492

Intermedio (índice 164)	checking_status = no checking	+0.2408
p(good) = 0.504	duration = 36	−0.1538
	credit_amount = 909	−0.1064
	purpose = new car	−0.0736
	age = 36	+0.0437

Tabla 12. Resumen del ajuste de LIME: g(x) es la predicción del modelo lineal en la propia instancia (intercepto + suma de los cinco coeficientes).

Cliente	**Intercepto**	**g(x) con K = 5**	**p(good) real**	**R^2_w (K = 5)**	**R^2_w (20 var.)**	**N_{ef}**
Aprobado	0.648	0.762	0.811	0.197	0.208	4808
Rechazado	0.630	0.340	0.194	0.213	0.244	4808
Intermedio	0.635	0.586	0.504	0.279	0.294	4808

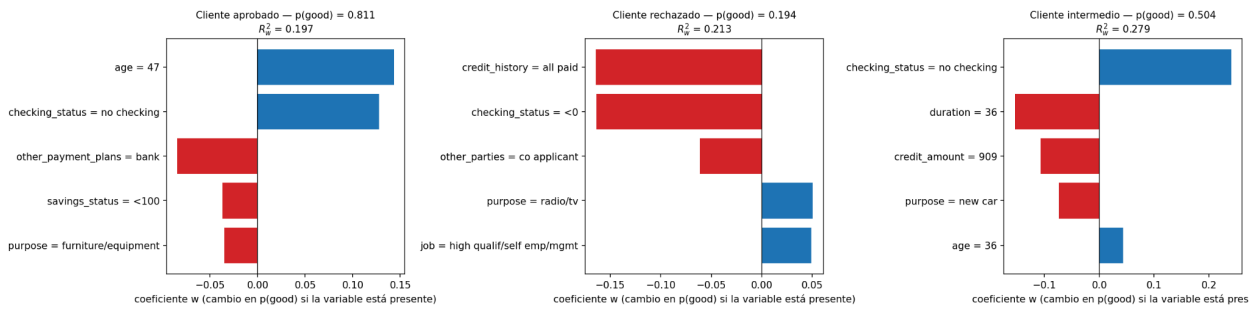


Figura 10. Coeficientes de LIME (K = 5) de los tres clientes.

Las variables principales tienen sentido: el estado de la cuenta corriente aparece en los tres clientes, y en el rechazado LIME coincide con SHAP en que *credit_history* = «all paid» y *checking_status* = «<0» son las dos razones principales. Sin embargo, la fidelidad es baja: el modelo lineal explica solo entre el 20 % y el 28 % de la varianza ponderada de p(good) en las perturbaciones, y en la propia instancia se equivoca por entre 0.05 y 0.15 (para el rechazado predice 0.340 cuando el modelo da 0.194). La causa es que el vecindario no es local: con σ_{rel} = 0.75 el peso π varía poco (N_{ef} = 4808 de 5000 muestras, el 96 %) y en promedio cada muestra reemplaza 10 de las 20 variables, así que g intenta aproximar con una recta un modelo de árboles con escalones e interacciones sobre una región muy amplia.

Task 3.2 — Estabilidad de LIME en el cliente rechazado

Para cada combinación de N ∈ {500, 2000, 5000} y σ_{rel} ∈ {0.5, 0.75, 1.5} se ejecutó LIME 20 veces con las semillas 42 a 61. Las Tablas 13 y 14 muestran el Jaccard promedio de los conjuntos top-5 sobre los 190 pares de corridas y el tamaño efectivo de muestra promedio $N_{ef} = (\Sigma_n \pi_n)^2 / \Sigma_n \pi_n^2$.

Tabla 13. Jaccard promedio entre los conjuntos de las K = 5 variables principales (20 corridas).

N	σ_{rel} = 0.5	σ_{rel} = 0.75	σ_{rel} = 1.5
500	0.455	0.449	0.447
2000	0.622	0.625	0.634
5000	0.706	0.693	0.692

Tabla 14. Tamaño efectivo de muestra promedio N_{ef}. La fila teórica es $E[\pi]^2/E[\pi^2]$ con $D^2 \sim$ Binomial(20, 0.5).

N	σ_{rel} = 0.5	σ_{rel} = 0.75	σ_{rel} = 1.5
500	409.7	480.5	498.8
2000	1639.4	1922.5	1995.1
5000	4099.9	4806.3	4987.7
N_{ef} / N medido	**0.820**	**0.961**	**0.998**
N_{ef} / N teórico	**0.821**	**0.961**	**0.998**

El Jaccard sube con N (con σ_{rel} = 0.75: 0.449, 0.625 y 0.693) y casi no depende de σ_{rel}. N_{ef} es una fracción fija de N para cada σ_{rel} y coincide con el valor teórico $E[\pi]^2/E[\pi^2]$, que se puede calcular de forma exacta porque D^2 sigue una binomial(20, 0.5): con un ancho relativo de 0.5 a 1.5 el kernel es casi plano y se pierde entre el 0.2 % y el 18 % de las muestras. Con N = 5000 y σ_{rel} = 0.75, *checking_status* y *credit_history* entran al top-5 en las 20 corridas, pero los otros tres lugares rotan entre *job* (19), *other_parties* (16), *purpose* (13), *duration* (4), *employment* (4), *other_payment_plans* (3) y *savings_status* (1). Los |w| medios del tercero al octavo lugar (0.060, 0.053, 0.048, 0.042, 0.040, 0.040) están separados por unos 0.005, menos que su variación entre corridas (desviación de 0.006 a 0.008): por eso el ruido de muestreo los intercambia.

Task 3.3 — LIME frente a SHAP

a. Correlación y coincidencia. Se comparó |w| de LIME (ajuste con las 20 variables) con |φ| de SHAP del Task 2.2. Como LIME explica la probabilidad y SHAP el log-odds, se comparan rangos y no magnitudes.

Tabla 15. Correlación de Spearman entre |w| (LIME) y |φ| (SHAP), y variables principales de cada método.

Cliente	Spearman	Top-3 LIME	Top-3 SHAP	Coincidencia
Aprobado	0.702	age, checking_status, other_payment_plans	checking_status, age, other_payment_plans	3/3
Rechazado	0.877	checking_status, credit_history, other_parties	credit_history, checking_status, duration	2/3
Intermedio	0.869	checking_status, duration, credit_amount	checking_status, duration, credit_amount	3/3

Los dos métodos coinciden bastante en el orden general (Spearman de 0.70 a 0.88) y en 8 de las 9 variables principales. La única diferencia está en el tercer lugar del rechazado: LIME pone *other_parties* = «co applicant» (φ = −0.39, 4.º en SHAP) y SHAP pone *duration* = 18 (−0.39), dos variables con |φ| prácticamente iguales.

b. Fidelidad según K. Con las mismas 5000 muestras (semilla 42) se ajustó LIME con K ∈ {3, 5, 10, 20}. El cliente pedido es el rechazado; los otros dos se agregan como referencia.

Tabla 16. Fidelidad ponderada R^2_w según el número de variables K.

K	Rechazado	Aprobado	Intermedio
3	0.196	0.186	0.259
5	0.213	0.197	0.279
10	0.237	0.205	0.292
20	0.244	0.208	0.294

Task 3.4 — Explicaciones contrafactuales del cliente rechazado

Siguiendo la diapositiva de contrafactuales, se buscó el cambio más cercano y accionable que lleve al cliente rechazado (p = 0.194) a la región de aprobación. Se generaron 20 000 candidatos; cada uno cambia un subconjunto aleatorio de 1 a 3 de las variables accionables (*duration*, *credit_amount*, *installment_commitment*, *checking_status* y *savings_status*) y deja fijas las inmutables (*age*, *personal_status* y *foreign_worker*) y todas las demás. El nuevo valor de cada variable se tomó al azar de los valores observados en entrenamiento, excluyendo el valor actual para que la variable realmente cambie. Un candidato es válido si p(good) ≥ θ = 0.5. En la distancia, las MAD de entrenamiento son 6.0 para *duration*, 1057.5 para *credit_amount* y 1.0

para *installment_commitment*; *existing_credits* y *num_dependents* tienen MAD = 0 y usan la desviación absoluta media (0.514 y 0.274), aunque no cambian en ningún candidato.

Tabla 17. Los tres contrafactuales seleccionados (valores decodificados).

Contrafactual	Variable cambiada	Original	Nuevo	Distancia	p(good)
1	checking_status	<0	no checking	1.000	0.611
2	checking_status + credit_amount	<0; 1940	no checking; 1943	1.003	0.611
3	duration	18	11	1.167	0.527

Se encontraron 6319 candidatos válidos, una validez de 0.316, así que no hizo falta subir el máximo de variables cambiadas a 5. La proximidad promedio de los tres contrafactuales es 1.057 y su dispersión promedio es 1.33 variables cambiadas (sobre todos los candidatos válidos: 2.81 y 2.33). *checking_status* aparece en el 79 % de los candidatos válidos, frente al 34 %–48 % de las demás variables.

Dos observaciones importantes. Primero, el contrafactual 2 es en la práctica el mismo que el 1: sube el monto en 3 DM (distancia 0.003) y la probabilidad no cambia (0.611). Cumple la regla de tener un conjunto de variables distinto, pero no aporta diversidad real; una regla mejor exigiría que cada contrafactual nuevo cambie una variable no usada antes o que cada cambio tenga un efecto mínimo en p. Segundo, el contrafactual 1 dice que el cliente sería aprobado si no tuviera cuenta corriente, un patrón conocido de German Credit. Es accionable en el papel, pero equivale a cerrar la cuenta para engañar al modelo, no a ser mejor pagador. El contrafactual 3, reducir el plazo de 18 a 11 meses (p = 0.527), es el único que describe un cambio con sentido en la solicitud.

Task 3.5 — Preguntas

a. Relación entre N_{ef} y el Jaccard. Según las Tablas 13 y 14, el Jaccard depende casi solo de N_{ef}: los puntos de los tres σ_{rel} caen sobre la misma curva (Figura 11), creciente y cóncava en escala logarítmica. σ_{rel} cambia la estabilidad solo a través de N_{ef}, y como el kernel es casi plano ese efecto es pequeño.

Para la extrapolación a J = 0.9 con $\sigma_{rel} = 0.75$ se usó un modelo justificado por la forma del estimador. Cada w_j es un estimador de mínimos cuadrados ponderados, así que su error estándar decrece como $1/\sqrt{N_{ef}}$ y, por el teorema del límite central, su distribución entre corridas es aproximadamente normal. Con la media y la desviación de los 20 coeficientes medidos con N = 5000, se simularon corridas con $w_j \sim N(m_j, s_j^2 \cdot N_{ef}(5000)/N_{ef}(N))$, donde m_j y s_j son esa media y esa desviación, y se calculó el Jaccard esperado para cada N. Antes de extrapolar, el modelo se validó contra los valores medidos: predice 0.459, 0.604 y 0.721 frente a 0.449, 0.625 y 0.693 para N = 500, 2000 y 5000, con un error máximo de 0.03.

Tabla 18. Extrapolación y verificación del tamaño de muestra para un Jaccard de 0.9 (σ_{rel} = 0.75). La brecha es entre *purpose* (5.º) y *duration* (6.º), estimada con las corridas de cada etapa.

Paso	**N**	**N_{ef}**	**J predicho**	**J medido (20 corridas)**	**Brecha |w| 5.º–6.º**
Etapa 1 (desde N = 5000)	18 500	17 785	0.907	0.882	0.0058 ± 0.0019
Etapa 2 (desde N = 18 500)	24 000	23 072	0.910	**0.967**	0.0052 ± 0.0010

La primera estimación (18 500 muestras) quedó apenas por debajo al verificarla (0.882), porque con solo 20 corridas la brecha entre el 5.º y el 6.º lugar tiene un error estándar de 0.0019, un tercio de su valor. Al re-estimar con los coeficientes de la verificación, más precisos (0.0052 ± 0.0010), el modelo pidió 24 000 muestras, y la verificación dio J = 0.967 ≥ 0.9. Por lo tanto se necesitan unas 24 000 muestras ($N_{ef} \approx 23\,000$), unas 4.8 veces las 5000 actuales. La brecha se mantuvo estable (0.0058, 0.0052 y 0.0056), lo que confirma que *purpose* y *duration* no están empatadas y que el Jaccard sí puede llegar a 1; con un empate exacto en el 5.º lugar se quedaría en $0.5 \cdot 1 + 0.5 \cdot 4/6 \approx 0.83$ sin importar N. Como comparación, un ajuste de ley de potencias $1 - J = 2.67 \cdot N_{ef}^{(-0.257)}$ con los tres puntos medidos estimaría 376 000 muestras, 16 veces más: no sirve para extrapolar porque la probabilidad de que dos variables se crucen cae mucho más rápido que una potencia cuando el ruido se vuelve menor que su brecha.

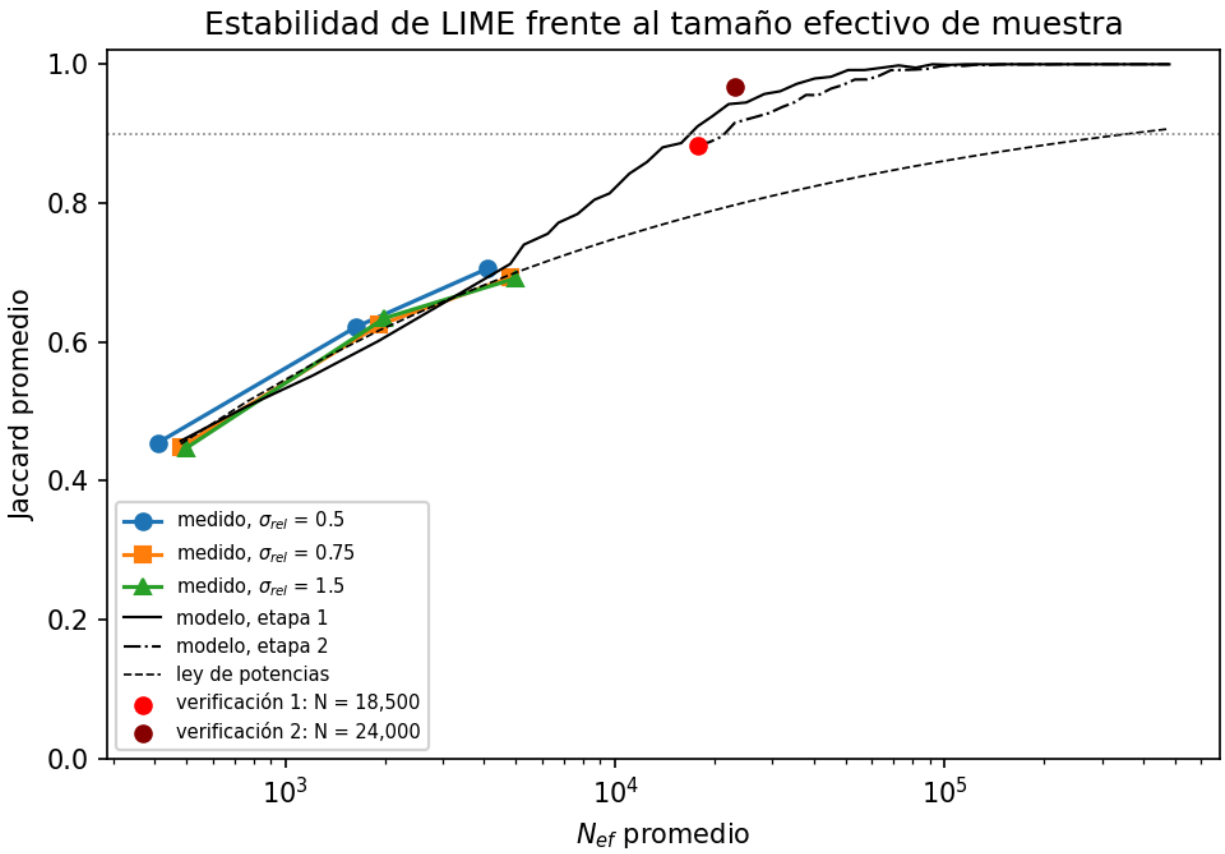


Figura 11. Jaccard promedio frente a N_{ef}, modelos de extrapolación y las dos verificaciones.

b. Mayor discrepancia entre LIME y SHAP. Para comparar escalas distintas, cada vector se normalizó por su suma de valores absolutos ($s_j = w_j/\Sigma|w|$ y $s_j = \varphi_j/\Sigma|\varphi|$), y la discrepancia se midió como $|s_{LIME} - s_{SHAP}|$, que tiene en cuenta signo y magnitud. La mayor está en el cliente aprobado, variable duration = 18: SHAP le da $\varphi = -0.208$ (5.º lugar, $s = -0.056$), mientras que LIME le da $w = +0.011$ (12.º lugar, $s = +0.019$), con el signo contrario.

Hipótesis: la discrepancia no se debe a la localidad sino a que la respuesta del modelo a *duration* es no lineal alrededor de 18 meses, de modo que el efecto depende de con qué valores se compare y de qué otras variables estén presentes. Se pusieron a prueba las tres explicaciones posibles:

- **Localidad (no se confirma).** Con $\sigma_{rel} \in \{0.25, 0.5, 0.75, 1.0, 1.5\}$ y 5 semillas cada uno, el s medio de LIME fue 0.002, 0.005, 0.005, 0.006 y 0.006 (desviación de 0.012 a 0.025) y la discrepancia se mantuvo entre 0.058 y 0.062. Reducir el ancho del kernel no la baja. Además, el valor de LIME es indistinguible de cero: su media sobre semillas (0.005) es menor que su desviación (0.012).
- **No linealidad (se confirma).** La Figura 12 muestra que p(good) es una función escalonada de *duration*: el cliente está en una meseta entre 16 y 21 meses (p = 0.811); con plazos menores sube hasta 0.94 (+1.30 en log-odds) y con plazos mayores cae a 0.51

en 40 meses y a 0.21 en 72 meses (−2.76 en log-odds). Reemplazar 18 por un valor aleatorio produce ganancias y pérdidas de signo contrario que casi se cancelan. En log-odds pesan más las pérdidas por plazos largos y SHAP da −0.21; en probabilidad la sigmoide comprime las ganancias cerca de 0.9. SHAP intervencional sobre la probabilidad, con el mismo fondo, da φ = −0.011 (s = −0.022): la distancia a LIME baja de 0.075 a 0.041, pero el signo sigue siendo negativo.

- **Interacción (se confirma).** El aporte marginal promedio $\Delta_j(s) = E[f(x_{S \cup j}, r) - f(x_S, r)]$ cambia mucho con el tamaño de la coalición: en log-odds va de −0.33 (|S| = 2) a −0.06 (|S| = 15), cinco veces menos, y en probabilidad cambia de signo, de −0.025 con coaliciones pequeñas a +0.012 con |S| = 19. Su promedio sobre los tamaños (−0.200) reproduce el φ de SHAP (−0.208). Además, el cambio por plazos cortos es mayor en este cliente (+1.30 en log-odds) que en promedio sobre el fondo (+0.85): el efecto de *duration* depende de los demás valores del cliente.

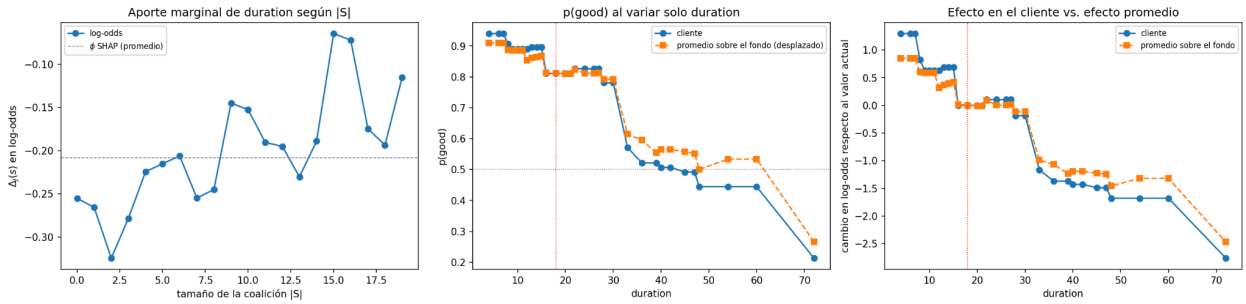


Figura 12. Cliente aprobado, variable duration: aporte marginal según |S|, p(good) al variar solo duration y efecto en el cliente frente al promedio sobre el fondo.

Conclusión: la discrepancia se debe a la no linealidad de la respuesta (escalones alrededor del valor del cliente y saturación de la sigmoide) combinada con una interacción con las demás variables, y no a la localidad. SHAP promedia el aporte con el mismo peso para cada tamaño de coalición y en log-odds, mientras que LIME ajusta una recta a la probabilidad con un kernel casi plano; con un efecto que cambia de signo según el contexto, una recta da un coeficiente cercano a cero.

c. Elección de K. Según la Tabla 16, el R^2_w del rechazado crece poco con K: 0.196, 0.213, 0.237 y 0.244. Elegiría K = 5. Por comprensibilidad, cinco razones es lo que una persona puede leer y recordar, y es el formato habitual de una notificación de rechazo; con 10 o 20 variables la explicación ya no se entiende. Por fidelidad, K = 5 conserva el 87 % del R^2_w que se lograría con las 20 variables (0.213 de 0.244), y pasar a 10 duplica las variables para ganar solo 0.024. K = 3 pierde poco (0.196), pero en el rechazado dejaría fuera a *purpose* y *job*, las dos razones a favor.

Para una decisión de crédito aceptaría como mínimo $R^2_w \geq 0.8$: si la explicación se usa para decirle a una persona por qué fue rechazada, el modelo sustituto debe reproducir la mayor parte del comportamiento del modelo real en su vecindario; con menos, las razones pueden ser las del sustituto y no las del modelo. Aquí ni siquiera con las 20 variables se pasa de 0.244, así que no

entregaría la explicación de LIME en esta configuración. Si no se alcanza, haría lo siguiente: (1) volver más local el vecindario (σ_{rel} menor y perturbaciones con valores cercanos a los del cliente en lugar de valores aleatorios de toda la columna) y medir de nuevo R^2_w; (2) si sigue sin llegar, usar una explicación con garantías para este modelo, como TreeSHAP, que es exacto (error de eficiencia de 9.7×10^{-8}), o contrafactuales verificados con el propio modelo; y (3) en decisiones de alto riesgo, considerar un modelo interpretable por diseño, como sugiere Rudin (2019).

d. Contrafactuales frente a SHAP. Las tres variables de mayor |φ| del rechazado son *credit_history* (−0.94), *checking_status* (−0.84) y *duration* (−0.39); los contrafactuales cambian *checking_status*, *duration* y *credit_amount*. La Tabla 19 mide el mejor resultado posible cambiando una sola variable.

Tabla 19. Efecto de cambiar una sola variable en el cliente rechazado (p = 0.194).

Variable	Tipo	φ	Mejor p sola (valor)	Dist. mínima a θ	Máx. Δp por unidad	En CF
credit_history	no accionable	−0.938	0.527 (critical/other)	1.00	0.333	No
checking_status	accionable	−0.837	0.611 (no checking)	1.00	0.416	Sí
duration	accionable	−0.394	0.549 (4 meses)	1.17	0.487	Sí
credit_amount	accionable	−0.042	0.512 (392 DM)	1.46	0.221	Sí
installment_commitment	accionable	−0.074	0.207 (2)	no llega	0.013	No
savings_status	accionable	−0.170	0.327 (no known)	no llega	0.133	No

La variable de |φ| alto que no aparece en los contrafactuales es credit_history, la más importante para SHAP. No falta por costo ni por efecto pequeño: cambiarla a «critical/other existing credit» costaría lo mismo que *checking_status* (distancia 1) y bastaría para aprobar (p = 0.527). Falta porque no es accionable: es el historial de pagos pasados, que el cliente no puede modificar, y no está en la lista de variables que la búsqueda puede cambiar. Los otros dos casos muestran las otras razones. *installment_commitment* es accionable, pero su efecto por unidad es pequeño (0.013 de probabilidad por unidad de distancia) y nunca alcanza θ sola. *savings_status* tiene un φ moderado (−0.17), pero ningún valor la lleva por encima de 0.33. Al revés, *credit_amount*

aparece en un contrafactual a pesar de tener un φ casi nulo (−0.04), solo como relleno con un cambio de 3 DM.

SHAP responde **¿cuánto contribuyó cada variable a que la predicción de este cliente se aleje del promedio?**: reparte la diferencia $f(x) - \varphi_0$ entre las variables, sin importar si se pueden cambiar. El contrafactual responde **¿qué es lo mínimo que tendría que cambiar para que la decisión fuera otra?**: depende del umbral θ, de qué variables se permite mover y de cómo se mide el costo. Una variable puede pesar mucho en SHAP y no servir como recomendación (*credit_history*), y una con φ pequeño puede ser parte de la receta más barata.

Task 4 — Validación de las explicaciones

Task 4.1 — Deletion e insertion

Se validó el mapa de Grad-CAM de 224 × 224 del Task 1.2 en las tres imágenes, para la clase de mayor probabilidad, siguiendo el panel B de la diapositiva «Desconfiar de las explicaciones». Los píxeles se ordenaron por el valor del mapa de mayor a menor y, en P = 20 pasos, se modificó la fracción k/P de los más importantes (los tres canales de cada píxel). En deletion se reemplazaron por gris medio (0.5 por canal antes de normalizar); en insertion se partió de la imagen desenfocada (gaussiano con σ = 10 px y kernel de 61 px) y se insertaron los píxeles originales. En cada caso $AUC = (1/P)\ \Sigma_k\ c_k$, con c_k la probabilidad de la clase en el paso k. Se repitió todo con 10 órdenes aleatorios de píxeles (semilla 42), los mismos para las tres imágenes y las dos métricas. En deletion un AUC bajo es mejor; en insertion, uno alto.

Tabla 20. Deletion e insertion del Grad-CAM frente a 10 órdenes aleatorios (media ± desviación). z = diferencia / desviación del azar. Probabilidad original de la clase: 0.997, 0.233 y 0.421.

Imagen (clase)	AUC_{del} mapa	AUC_{del} azar	Dif. (z)	AUC_{ins} mapa	AUC_{ins} azar	Dif. (z)
(a) banana	0.1493	0.0364 ± 0.0079	+0.113 (+14.3)	0.7712	0.1154 ± 0.0044	+0.656 (+148.8)
(b) Egyptian cat	0.0002	0.0007 ± 0.0005	−0.0004 (−0.8)	0.2275	0.0123 ± 0.0003	+0.215 (+643.0)
(c) ox	0.0034	0.0162 ± 0.0039	−0.013 (−3.3)	0.3845	0.0609 ± 0.0062	+0.324 (+52.3)

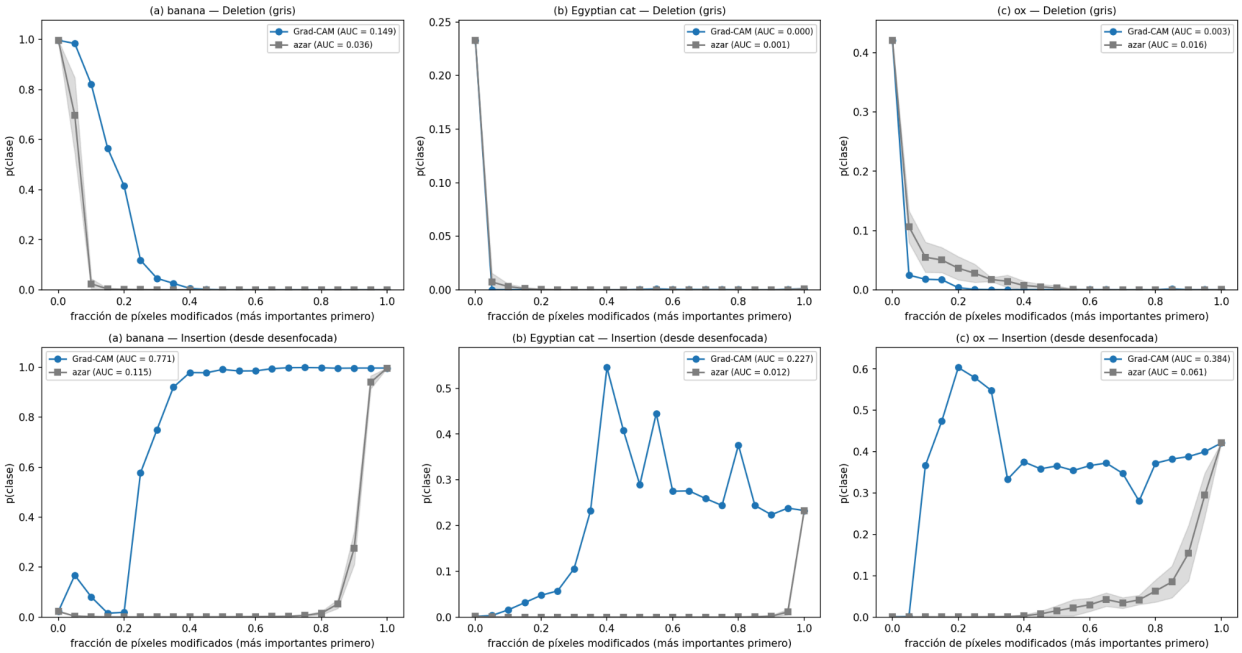


Figura 13. Curvas de deletion (gris) e insertion (desde la imagen desenfocada) del Grad-CAM y del azar (media ± desviación).

Task 4.2 — Aleatorización de parámetros en cascada

Siguiendo el panel A de la misma diapositiva, sobre una copia del modelo se reinicializaron con la inicialización por defecto de PyTorch (reset_parameters(): Kaiming uniforme en Conv2d y Linear; peso 1, sesgo 0 y estadísticas 0/1 en BatchNorm2d), de forma acumulativa, los pesos de fc, luego layer4, layer3, layer2, layer1 y conv1 (bn1 conserva sus pesos, como pide el enunciado), con torch.manual_seed(42) antes de la cascada. En cada etapa se calculó el Grad-CAM de 7 × 7 para la clase de mayor probabilidad del modelo original y su correlación de Spearman con el mapa original.

Tabla 21. Spearman entre el Grad-CAM 7 × 7 de cada etapa y el original, y la nueva clase más probable del modelo aleatorizado.

Etapa	(a) banana	(b) Egyptian cat	(c) ox	Nueva top-1 (a / b / c)
fc	0.811	−0.383	0.793	fountain pen / shield / mushroom
+ layer4	−0.346	0.243	−0.164	barracouta / brambling / barracouta
+ layer3	0.160	0.225	0.087	colobus / barracouta / barracouta
+ layer2	0.013	−0.140	−0.192	colobus / colobus / colobus
+ layer1	0.244	−0.193	−0.161	colobus / colobus / colobus
+ conv1	0.229	0.050	0.212	colobus / colobus / colobus

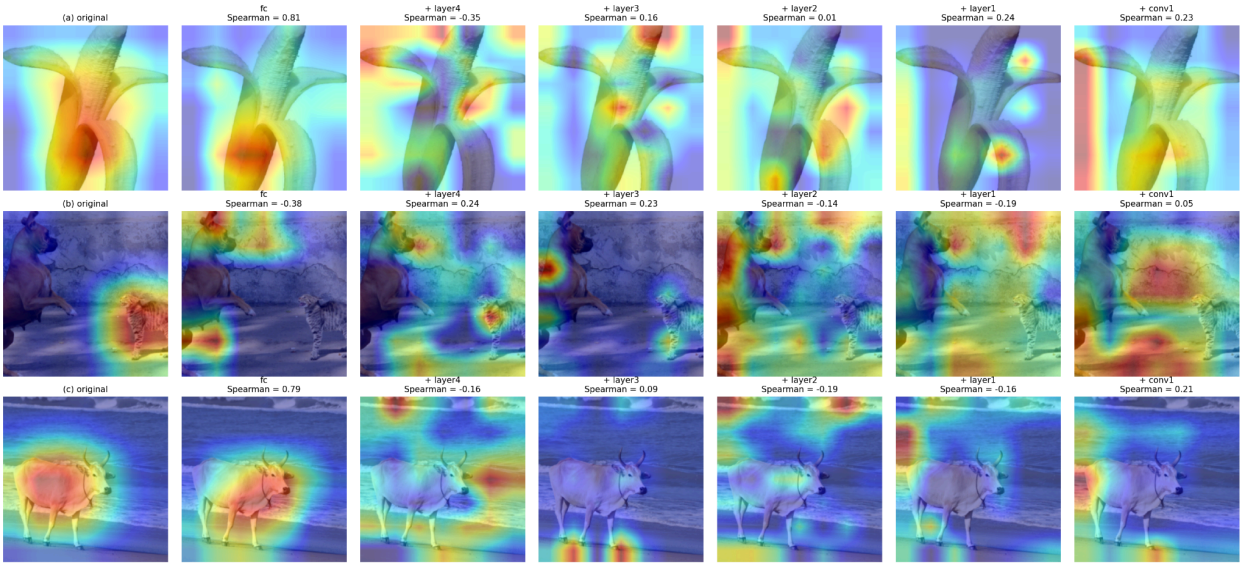


Figura 14. Grad-CAM original y en cada etapa de la cascada para la clase original de cada imagen.

Desde layer2 el modelo aleatorizado predice *colobus* para las tres imágenes, señal de que ya no distingue entre entradas. Con los pesos sin ReLU los valores son parecidos (por ejemplo, −0.63 en la etapa fc de (b)).

Task 4.3 — Preguntas

a. ¿Supera Grad-CAM al azar? En insertion sí, en las tres imágenes y por mucho: el AUC del mapa supera al del azar por 0.66, 0.22 y 0.32, es decir, por 149, 643 y 52 desviaciones estándar. En deletion solo en (c), por 3.3 desviaciones; en (b) la diferencia no es significativa (−0.8 desviaciones) y en (a) el mapa es peor que el azar por 14.3 desviaciones. Los z son enormes en parte porque la desviación del azar es muy pequeña (0.0003 a 0.008); la diferencia de AUC es la medida de efecto más útil.

La forma de las curvas se explica por las 7 × 7 posiciones del mapa original. Dentro de cada celda de 32 × 32 píxeles el orden lo decide la interpolación bilineal, no la importancia real de cada píxel, así que el mapa quita o inserta manchas compactas alrededor de las mejores celdas: el 5 % de los píxeles (2509, el equivalente a 2.5 celdas) toca solo de 4 a 7 de las 49 celdas, el 25 % toca 18 o 19, y las 49 se completan recién al final. Por eso las curvas del mapa avanzan por saltos cuando se completa un bloque decisivo del objeto. En la insertion de (a), p(banana) pasa de 0.02 al 20 % a 0.58 al 25 % y a 0.98 al 40 %; en deletion se mantiene en 0.98 al 5 % y cae a 0.12 al 25 %. El orden aleatorio, en cambio, reparte los píxeles en las 49 celdas desde el primer paso (unos 51 píxeles por celda), lo que crea un patrón de alta frecuencia que la red nunca vio: en deletion destruye la predicción en un 5 %–10 % (en (a), de 0.997 a 0.70 y luego a 0.02), y en insertion la

imagen sigue viéndose desenfocada hasta el 85 %–90 %. Así, el azar es una referencia muy fuerte en deletion y muy débil en insertion.

Las insertions de (b) y (c) superan la probabilidad original: con el 40 % de los píxeles del gato y el resto desenfocado, p(Egyptian cat) llega a 0.55, frente a 0.23 en la imagen completa, y con el 20 % de la vaca p(ox) llega a 0.60, frente a 0.42. Al insertar solo el objeto, el contexto que compite queda desenfocado (el perro en (b) y la playa en (c)), lo que es coherente con el Task 1.4c, donde la playa le restaba confianza a *ox*. En (b), la deletion no puede discriminar por un efecto piso: la probabilidad parte de 0.23 y con un 5 % de píxeles cambiados cae a menos de 0.01 tanto con el mapa como con el azar.

Efecto de los parches grises. Se repitió la deletion de la imagen (a) con la imagen desenfocada como referencia (Tabla 22 y Figura 15).

Tabla 22. Deletion con referencia gris frente a desenfocada. La pedida es la imagen (a); (b) y (c) se agregan como referencia.

Imagen	**AUC_{del} mapa (gris)**	**AUC_{del} mapa (desenfocada)**	**AUC_{del} azar (gris)**	**AUC_{del} azar (desenfocada)**	**z (desenfocada)**
(a)	0.149	**0.302**	0.036 ± 0.008	0.067 ± 0.012	+20.4
(b)	0.0002	0.0002	0.0007 ± 0.0005	0.0005 ± 0.0003	−1.2
(c)	0.003	0.013	0.016 ± 0.004	0.043 ± 0.014	−2.2

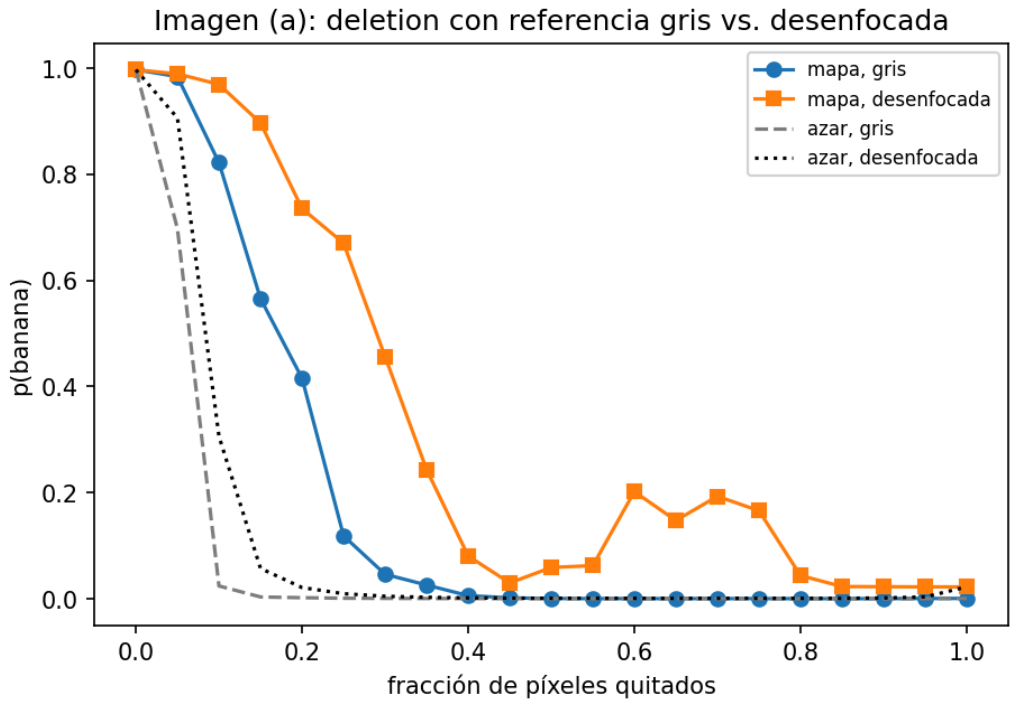


Figura 15. Imagen (a): deletion con referencia gris frente a desenfocada, para el mapa y para el azar.

Con la referencia desenfocada, el AUC_{del} del mapa en (a) se duplica (0.149 a 0.302): al quitar el 25 % más importante p(banana) sigue en 0.67, frente a 0.12 con gris. El parche desenfocado conserva el color amarillo y la forma general del banano, así que el modelo lo sigue reconociendo; el parche gris, en cambio, elimina además el color y crea un hueco con forma de objeto que es en sí mismo una señal fuera de distribución. Parte de la caída que se ve con gris se debe al parche y no a la información que se quitó. El orden aleatorio también sube (0.036 a 0.067), pero sigue destruyendo la imagen mucho antes que el mapa, de modo que en (a) Grad-CAM queda peor que el azar con cualquiera de las dos referencias.

Conclusión: deletion con píxeles dispersos mide sobre todo la sensibilidad de la red a artefactos, y su resultado depende de la referencia elegida. Aquí insertion es la prueba más informativa, y la que respalda claramente al mapa.

b. Cascada. El Spearman cae por debajo de 0.3 en + layer4 en las imágenes (a) y (c) (de 0.81 y 0.79 a −0.35 y −0.16), y desde la primera etapa, fc, en la imagen (b) (−0.38). Desde layer4 todos los valores quedan entre −0.35 y 0.25, alrededor de cero: Grad-CAM pasa la prueba, porque su mapa depende de los parámetros aprendidos y no solo de la imagen.

Predicción para la etapa fc. Como $\alpha_k^c = w_k^c/Z$, al aleatorizar solo fc las activaciones A^k de layer4 no cambian y los α pasan a ser los nuevos pesos aleatorios divididos entre 49. Se verificó: $|\alpha - \tilde{w}/Z| \leq 1.7 \times 10^{-10}$ y |A nueva − A original| = 0 en las tres imágenes. El mapa queda como $\mathrm{ReLU}(\Sigma_k\ \tilde{w}\ A^k)/Z$, una combinación lineal aleatoria, con pesos de media cero, de los mismos 512

mapas de activación. Predicción: en promedio debería perder la correlación con el mapa original, pero con mucha variación entre sorteos, porque los mapas A^k comparten estructura espacial (muchos canales se activan sobre el objeto). Para cuantificarla se simularon 500 inicializaciones aleatorias de fc, lo que no requiere retropropagar.

Tabla 23. Spearman de la etapa fc frente a 500 inicializaciones aleatorias de fc (un mapa constante ocurre cuando la ReLU anula todas las posiciones).

Imagen	Spearman etapa fc	Simulación: media	Percentil 5	Percentil 95	% mapas constantes
(a) banana	0.811	0.144	−0.465	0.793	5.8
(b) Egyptian cat	−0.383	0.079	−0.503	0.726	1.8
(c) ox	0.793	0.145	−0.591	0.813	3.2

El resultado es coherente con la predicción. Con pesos aleatorios, el Spearman esperado es bajo (0.08 a 0.15), pero cualquier valor entre −0.5 y 0.8 es plausible. El −0.38 de (b) está dentro de lo esperado; los 0.81 y 0.79 de (a) y (c) están en el extremo superior de la distribución (por encima del percentil 95 en (a) y justo debajo en (c)), es decir, fueron sorteos con suerte en los que la combinación aleatoria volvió a resaltar el objeto. Por eso, con la relación $\alpha_k^c = w_k^c/Z$, la primera etapa de la cascada es una prueba débil si se hace con una sola semilla: aleatorizar fc equivale a elegir una dirección de clase al azar sobre las mismas activaciones. La prueba decisiva es la de layer4, donde cambian las propias A^k.

c. Explicaciones para tres audiencias. La Tabla 24 resume qué entregaría del modelo de crédito del Task 2 a cada audiencia, con qué validación y qué no entregaría.

Tabla 24. Explicaciones del modelo de crédito según la audiencia.

Audiencia	Qué entregaría	Validación que lo respalda	Qué no entregaría
Persona solicitante	Un contrafactual accionable y razonable (reducir el plazo de 18 a 11 meses → p = 0.527) y las 2–3 razones principales en lenguaje simple, a partir de SHAP (saldo negativo en la cuenta corriente, historial de crédito), revisadas por una persona.	Validez comprobada con el propio modelo ($p \geq 0.5$), una sola variable cambiada, variables inmutables fijas. SHAP exacto para árboles: error de eficiencia de 9.7×10^{-8}.	LIME ($R^2_w = 0.21$; con N = 5000 su top-5 cambia entre corridas, J = 0.69). Contrafactuales que invitan a manipular el modelo («no tener cuenta corriente») o triviales (subir 3 DM). Números en log-odds.
Oficial de cumplimiento interno	SHAP local (cascadas) y global (I_j, beeswarm) junto con la importancia por permutación; el conjunto de contrafactuales con sus métricas; LIME solo como apoyo, con N y fidelidad reportados.	Eficiencia de SHAP (9.7×10^{-8}) y coincidencia con el Shapley exacto (2.5×10^{-8}); KernelSHAP vs. TreeSHAP (MAE 0.004); SHAP vs. permutación (Spearman 0.65); validez 0.316, proximidad 1.06 y dispersión 1.33 de los contrafactuales; LIME: J = 0.69 con N = 5000 y 0.97 con N = 24 000.	Interpretaciones causales de SHAP: con variables correlacionadas el crédito se reparte arbitrariamente (Task 2.3), y *age* mueve mucho la salida (6.ª en I_j) sin aportar AUC (última por permutación).

Regulador	Documentación global: desempeño (AUC 0.765), importancias globales, el papel de las variables sensibles (*age* I_j = 0.289, *personal_status* 0.119, *foreign_worker* 0.043) y cómo se tratan (inmutables en los contrafactuales), metodología de validación y sus limitaciones.	Las pruebas de eficiencia, estabilidad (Jaccard y N_{ef}) y fidelidad ($R^2_w \leq 0.29$) de este informe, y la evidencia de que las explicaciones dependen de decisiones de diseño (fondo, codificación ordinal, referencia en deletion).	Explicaciones individuales con datos personales de clientes, y explicaciones sin validar (LIME con R^2_w bajo) presentadas como si fueran la lógica del modelo.

Riesgo de un contrafactual en detección de lavado. En un sistema de detección de lavado de dinero en remesas como el del Proyecto 2, entregar a la persona alertada un contrafactual como el del Task 3.4 («si cambiara esto, la transacción no generaría alerta») sería un riesgo grave, por tres razones. Primero, el contexto es adversarial: las variables de un sistema así (monto, frecuencia, número de beneficiarios, horario, canal) las controla quien lava dinero, así que el contrafactual se convierte en una receta de evasión. La búsqueda del Task 3.4 está diseñada justamente para encontrar el cambio más barato (aquí, en promedio, 1.3 variables con distancia ≈ 1), y en lavado ese cambio mínimo sería, por ejemplo, fraccionar las remesas en montos menores al umbral (pitufeo). Segundo, informar a una persona que su operación es sospechosa o fue reportada está prohibido por la normativa contra el lavado (en Guatemala, la Ley contra el Lavado de Dinero u Otros Activos, Decreto 67-2001, que protege la confidencialidad de los reportes a la IVE). Tercero, como se vio con «no tener cuenta corriente», el contrafactual puede proponer cambios que engañan al modelo sin cambiar el riesgo real. En su lugar entregaría: al cliente, ninguna información sobre la lógica de detección; a los analistas y al oficial de cumplimiento, la explicación de cada alerta con atribuciones (SHAP sobre las variables y los pasos de la secuencia del remitente), trazable a las transacciones concretas y a la tipología que activó la alerta, con su estabilidad validada; y al regulador, el reporte de la operación sospechosa con los hechos y la documentación global del modelo. Los contrafactuales podrían usarse solo internamente, para medir qué tan cerca del umbral está un caso o qué tan fácil sería evadir el modelo.

Conclusiones

- En ResNet-18, Grad-CAM y CAM son el mismo mapa salvo el factor Z = 49, porque el *global average pooling* hace que $\partial y^c/\partial A^k_{ij} = w_k^c/Z$ en todas las posiciones; los errores medidos ($\approx 10^{-7}$ en float32, $\approx 10^{-16}$ en float64) son solo de redondeo.
- Grad-CAM distingue al gato y al perro de la imagen (b) (IoU = 0), pero su resolución de 32 × 32 píxeles limita la localización de objetos pequeños o cercanos.
- En la vaca en la playa, el mapa (74.5 % de su masa en el objeto) y la prueba causal (p(ox) = 0.0001 con solo el fondo) coinciden en que el modelo usa el objeto y no un atajo de fondo; el contexto inusual solo reduce su confianza.
- TreeSHAP intervencional cumple la eficiencia (9.7×10^{-8}) y coincide con el Shapley exacto escrito a mano (2.5×10^{-8}). KernelSHAP lo aproxima con un error medio de 0.004 log-odds, pero es entre 83 y 137 veces más lento, y el cálculo exacto es inviable con muchas variables.
- LIME escrito desde cero coincide con SHAP en el orden general (Spearman de 0.70 a 0.88; 8 de 9 variables principales), pero con este vecindario casi global su fidelidad es baja ($R^2_w \leq 0.29$ incluso con 20 variables) y su top-5 es inestable con N = 5000 (Jaccard 0.69). Se necesitan unas 24 000 muestras para un Jaccard de 0.9, estimación que se verificó (0.967).
- Los contrafactuales del cliente rechazado son baratos (validez 0.316, distancia ≈ 1, 1.33 variables), pero el más cercano propone no tener cuenta corriente, un cambio que engaña al modelo; la variable más importante para SHAP, credit_history, no aparece porque no es accionable.
- Grad-CAM supera ampliamente al azar en insertion (52 a 643 desviaciones estándar), pero deletion con píxeles dispersos queda dominada por artefactos fuera de distribución y por la referencia elegida (en (a), el AUC_{del} del mapa se duplica con la referencia desenfocada). En la cascada, el mapa pierde la correlación al aleatorizar layer4 (Spearman < 0.3), así que depende de los parámetros aprendidos; aleatorizar solo fc es una prueba débil, porque $\alpha_k^c = w_k^c/Z$ la reduce a una combinación aleatoria de las mismas activaciones.

Ninguna técnica sirve igual para todas las audiencias. Al solicitante, un contrafactual accionable y revisado; al oficial de cumplimiento, SHAP con sus validaciones numéricas; al regulador, la documentación global. En detección de lavado, un contrafactual sería una receta de evasión y no debe ent